



**ΟΙΚΟΝΟΜΙΚΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΑΘΗΝΩΝ**
ATHENS UNIVERSITY
OF ECONOMICS
AND BUSINESS

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΜΕΤΑΠΤΥΧΙΑΚΟ ΔΙΠΛΩΜΑ ΕΙΔΙΚΕΥΣΗΣ (MSc)

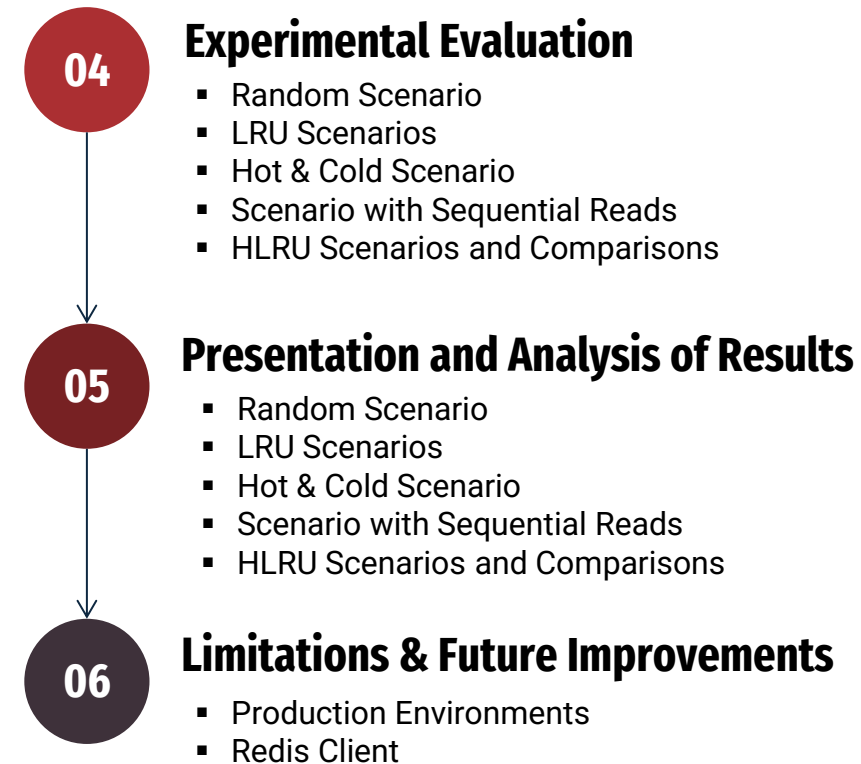
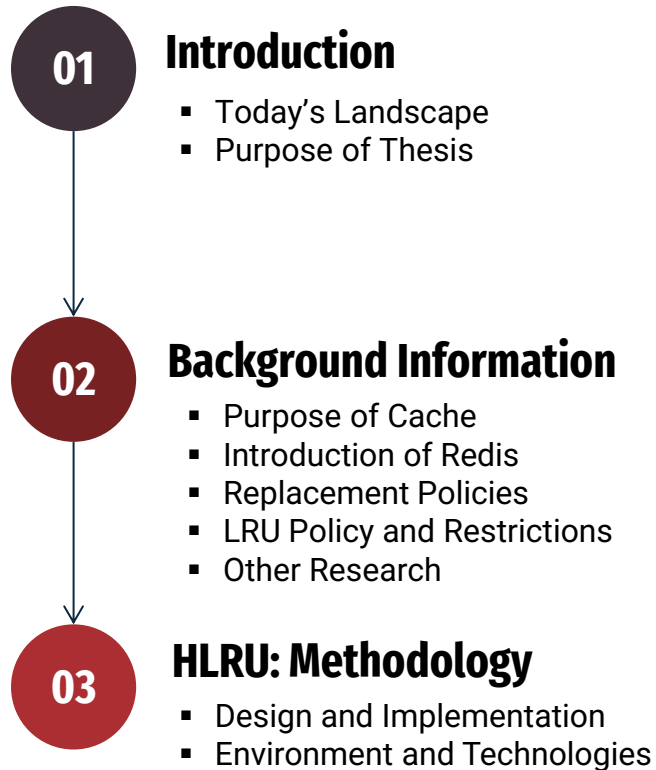
**στην ΑΝΑΠΤΥΞΗ & ΑΣΦΑΛΕΙΑ ΠΛΗΡΟΦΟΡΙΑΚΩΝ
ΣΥΣΤΗΜΑΤΩΝ**

**Αξιολόγηση και Ανάπτυξη
Νέων Πολιτικών Αντικατάστασης Κρυφής Μνήμης:
Η Πολιτική Histogram-based LRU**

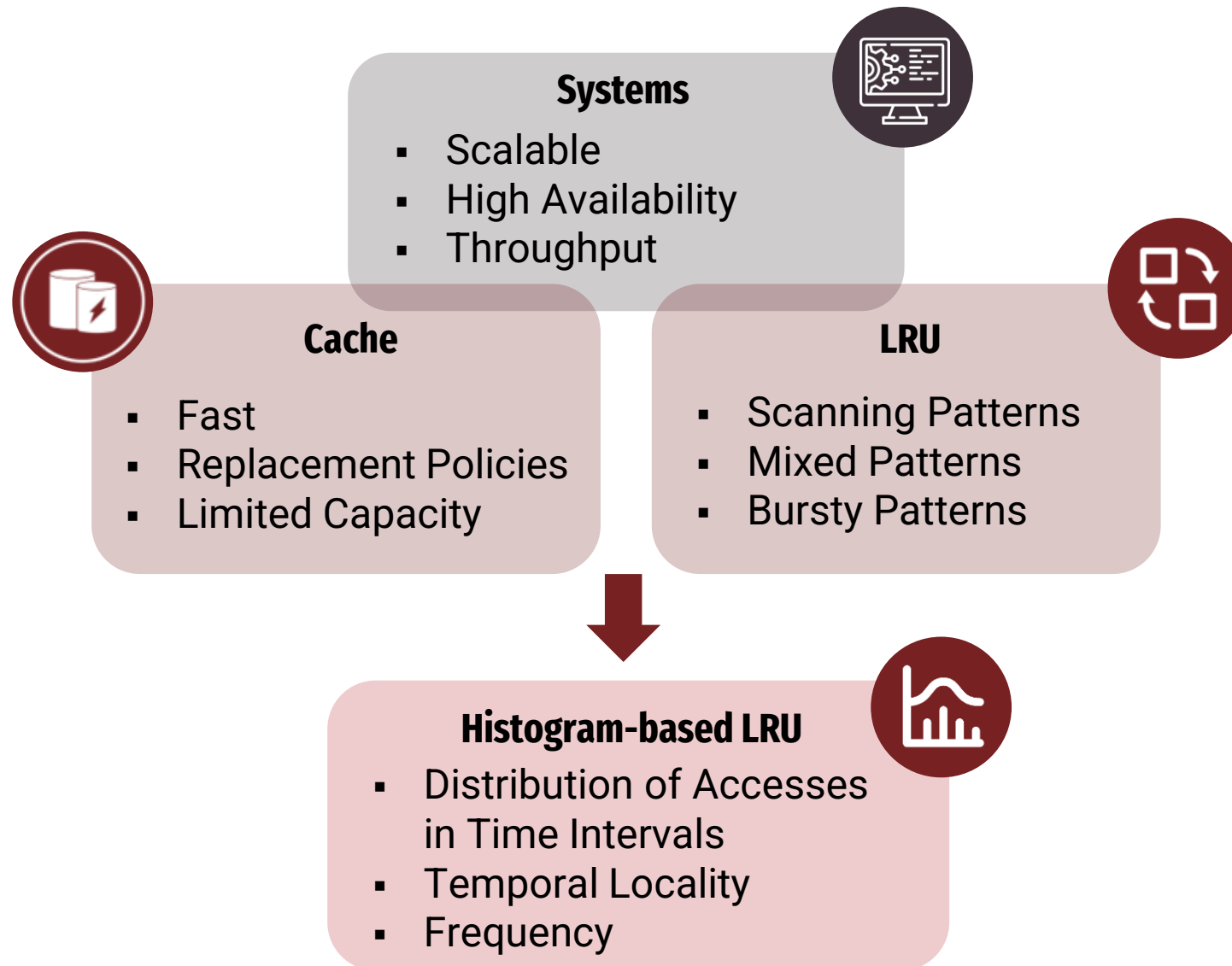
Δρίβα Ελένη

Αθήνα, Μάρτιος 2025

Outline



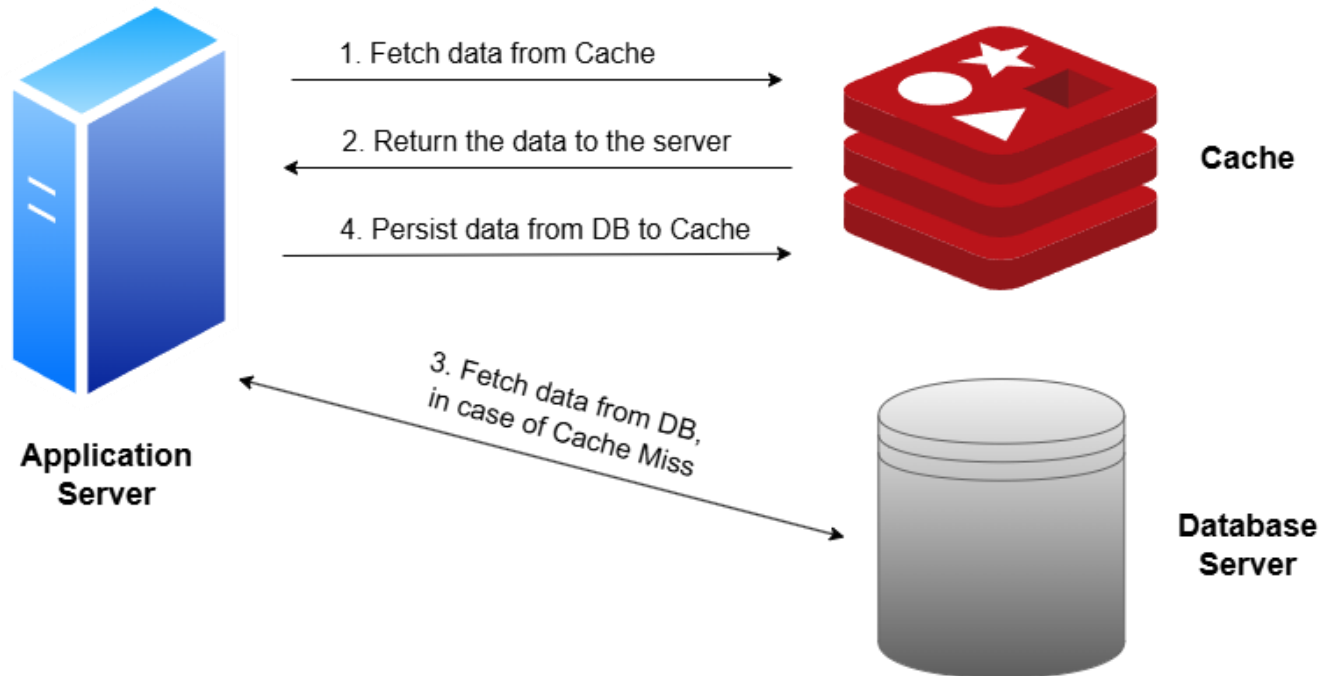
Introduction



Background Information

The Role of Caching in Modern Software Performance

How does Caching work?



Purpose

- Faster data retrieval
- Reduce traffic to critical resources
- Increase system's overall throughput

Recently or Frequently accessed data?
Let's get Caching!

Background Information

Remote Dictionary Server

Key Characteristics

- Open-source NoSQL Database
- Key-Value Pairs
- Multiple Uses
 - Cache Memory
 - Database
 - Message Broker
- Supports multiple data types
 - Strings
 - Hashes
 - Lists
 - Streams
 - ...

Applications

- Real Time Applications
 - Gaming
 - Financial Services
 - IoT

Users of Redis

- Twitter
- Airbnb
- Amazon
- OpenAI
- Tinder

Background Information

Redis - Replacement & Policies

Replacement Mechanism

1. **Data Insertion:** A **PUT** command adds new data to the cache.
2. **Memory Monitoring:** Redis tracks memory usage and triggers **eviction** when it exceeds **maxmemory**.
3. **Eviction Process:** Keys are removed based on the selected **replacement policy**.
4. **Continuous Cycle:** Cache memory surpasses **maxmemory**, and eviction restores it below the limit.

Policies

- Noeviction (*default*)
 - allkeys-lru → power-law distribution & unsure
 - allkeys-lfu → uniform distribution
 - allkeys-random
 - volatile-lru
 - volatile-lfu
 - volatile-random
 - volatile-ttl → we dictate the importance
- } → important keys & caching

Background Information

Redis - Approximated LRU

Grey: Evicted Keys

Dark grey: Not Evicted Keys

Red: Added Keys

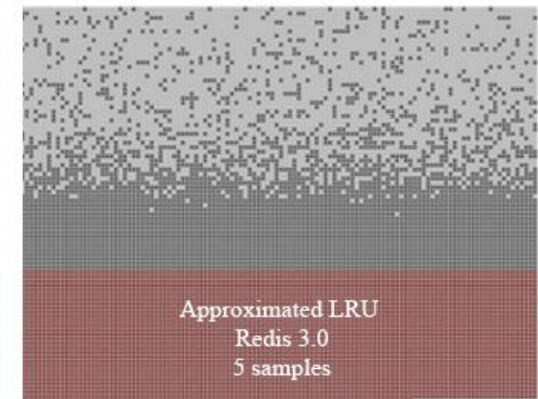
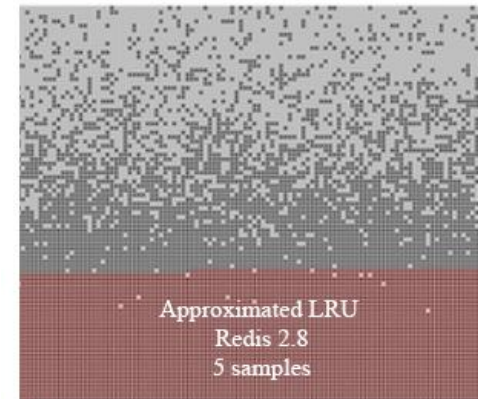
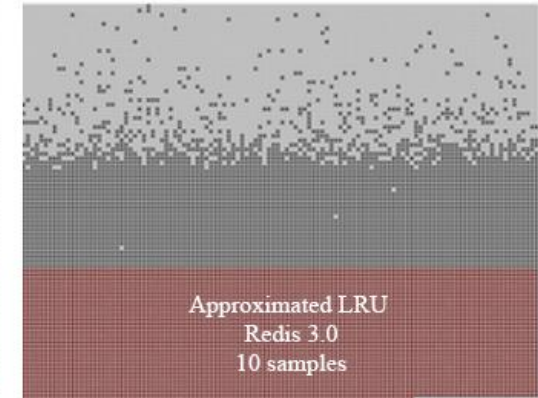
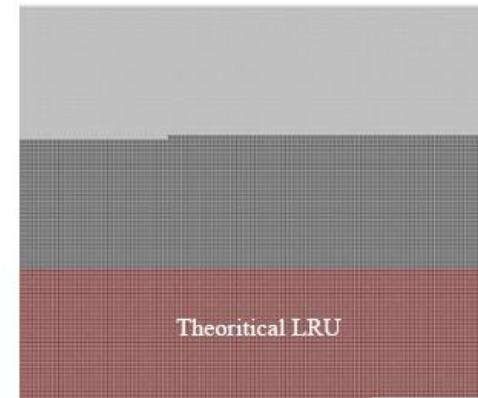
Theoretical LRU

- high **memory** overhead
- high **CPU** consumption

... instead:

Approximated LRU

- **Samples** a small **subset** of keys
- The oldest key in the sample is **evicted**
- The **accuracy** can be adjusted with *maxmemory-samples*



Background Information

LRU - Limitations

Burst Access Patterns

Data used periodically
Inability to predict future needs

Mixed Access Patterns

Data with different frequency
Inability to assess frequent data

Sequential Access Patterns

Data sequentially appearing
Constant needless refresh of data

Background Information

Other Research

ESCAPE: Elastic Caching for Big Data Systems

- elastic & scalable system
- dynamic addition/deletion of nodes
- Replacement: weighted factor on frequency and size of items

Penalty- and Locality-aware Memory Allocation in Redis Using Enhanced AET

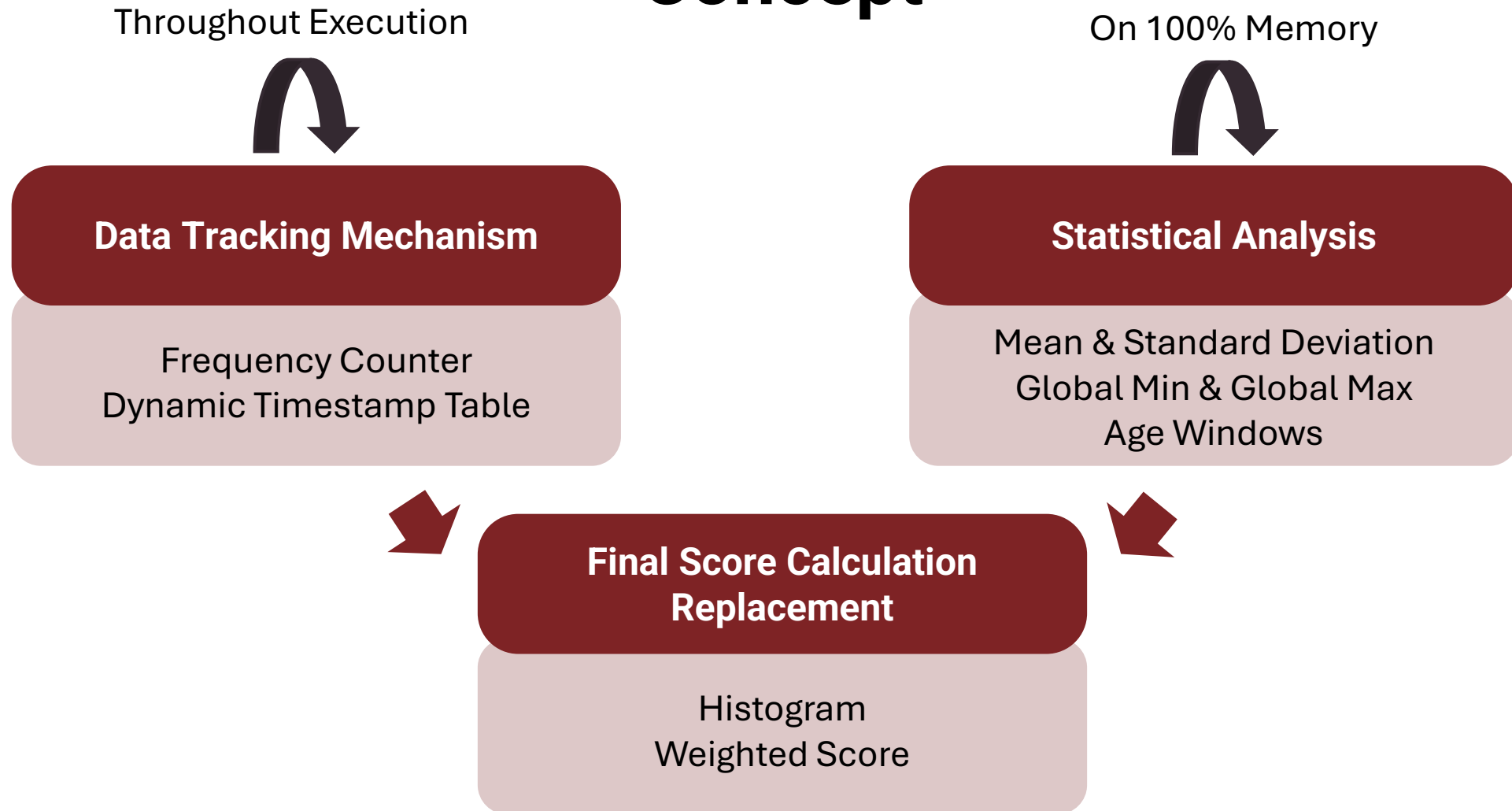
- proactive dump/load of data
- peak and off-peak hours
- Replacement: locality & miss penalty

Multi-Tenant In-memory Key-Value Cache Partitioning Using Efficient Random Sampling-Based LRU Model

- KRR algorithm models LRU sampling
- predicts cache performance more accurately
- dynamically adjusts memory allocation and optimises sample size

HLRU - Methodology

Concept



HLRU - Methodology

New Components

Dynamic Table (access_times)

Purpose

Holds the X latest access timestamps of each item

Requirements

24 bits (3 bytes) instead of 64 (8 bytes)*

Attributes

Configurable

Circular Rotation (keeps most recent values)

Empty Table of 5 capacity	—	—	—	—	—
Addition of 5 timestamps	1000	2000	3000	4000	5000
Addition of 6 th timestamp	2000	3000	4000	5000	6000

* Determining the relevant time difference of access times

Frequency Counter (num_accesses)

Purpose

Counter increasing each time an item is accessed

Requirements

8 bits (1 byte)

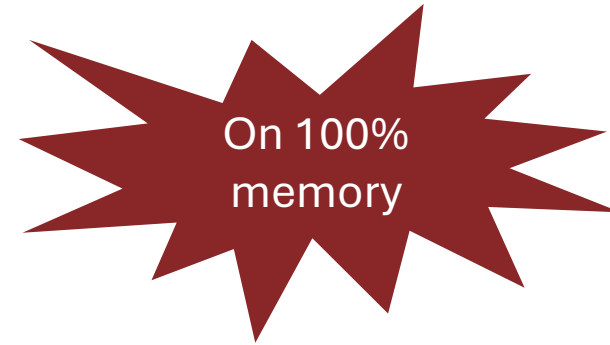
Attributes

Maintains precise frequency of access

Big or Small Table...?

HLRU - Methodology

Calculation of Statistics



Intermediary Computations

Program goes through all timestamps and calculates:

$$age = now - AT$$

- *Global Min* and *Global Max* ages are updated (from all sampled candidates)
- *Total Count* of accesses is computed



For each sampled candidate and each access time

Final Computations

When finished with all candidates, calculate *mean* and *standard deviation*:

$$mean = \frac{\sum age}{count}$$

$$stddev = \sqrt{\frac{\sum age^2}{count} - mean^2}$$

HLRU - Methodology

On 100%
memory

Calculation of Age Windows

1st window: $[global_{min}, \max(global_{min}, mean - stddev))$

2nd window: $[\max(global_{min}, mean - stddev), mean)$

3rd window: $[mean, \min(global_{max}, mean + stddev))$

4th window: $[\min(global_{max}, mean + stddev), global_{max}]$

From previously calculated

- Global Min & Global Max
- Mean & Standard Deviation

we construct the following **age windows**:

Why?

Dynamic Adaptation

Windows adjusted during each replacement procedure
Flexibility based on the data
Window Construction for both dense and sparse data

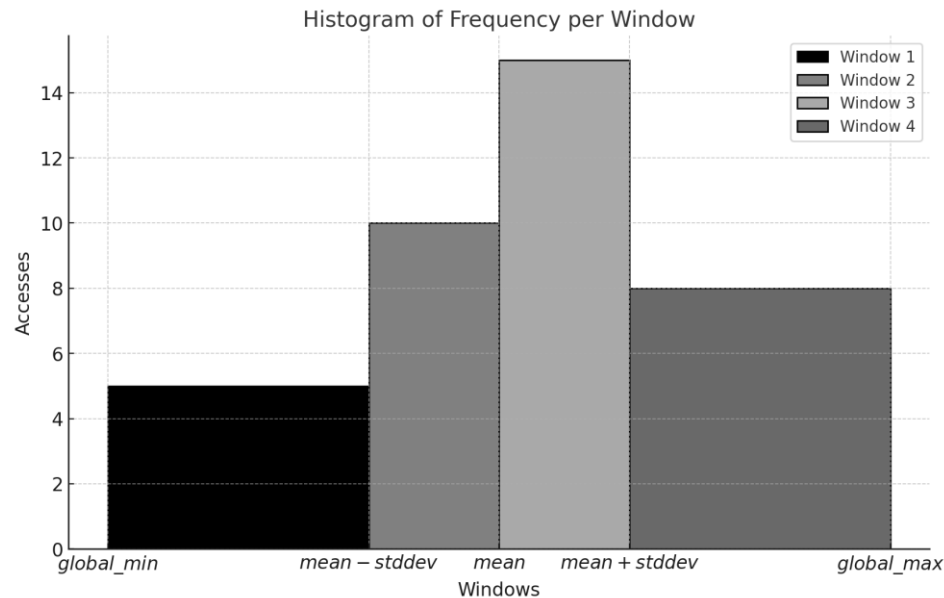
Boundary Preservation

Min and Max functions prevent value overflows for
precise data categorisation

HLRU - Methodology

On 100%
memory

Calculation of Final Score



Item Score

- **Creation of Histogram**
 - Age is calculated
 - Window_counts[i] increases
- **Corresponding Weight for each dynamic window**
- **Tie Breaker is added for ... ties!**

$$\text{score} = \sum_{i=1}^{\text{NUM_WINDOWS}} (\text{window_counts}[i] \times \text{WINDOW_WEIGHTS}[i]) + \text{tie_breaker}$$

Configurable
[0.4, 0.3, 0.2, 0.15]

0.01 x num_accesses

HLRU - Methodology

Environment

Infrastructure & Environment

- **Virtual Machine**
Ubuntu OS - 5 CPU Cores - 13.2 GB RAM
- **Redis Version**
7.2.x Release Version (including Redis 3.0)

Implementation

- **Client - Server Communication**
- **Java 11**
- **RESTful API**
- **Spring Boot 2.7.11**
- **Postgres** (*postgres: 14.2-alpine*)
- **Micrometer**

Tools for Experiments

- **Prometheus** (dockerised)
- **Redis Exporter** (dockerised)
- **Grafana** (dockerised)
- **JMeter**

Experimental Evaluation



Experiment Categories



LRU (*Redis Classic*) vs HLRU (*Redis Optis*)

- Random
- LRU-Favoured with $f > 1$
- LRU-Favoured with $f = 1$
- Hot & Cold
- Scan



HLRU (AT = {1, 2, 3, 4, 5})



Experimental Setup/Steps

- ◆ Cache Initialisation
- ◆ **Data Access Pattern**
- ◆ Eviction Condition Creation (+1/3 of data size)
- ◆ **Object Retrieval Pattern** (+1/3 of data size)



Key Parameters & Settings

- 📌 Cache Miss Penalty: 300 ms (configurable)
- 📌 Maxmemory-samples: 5 (default)
- 📌 WINDOW_WEIGHTS = {0.4, 0.3, 0.2, 0.15}
- 📌 Access Times (AT): Various configurations

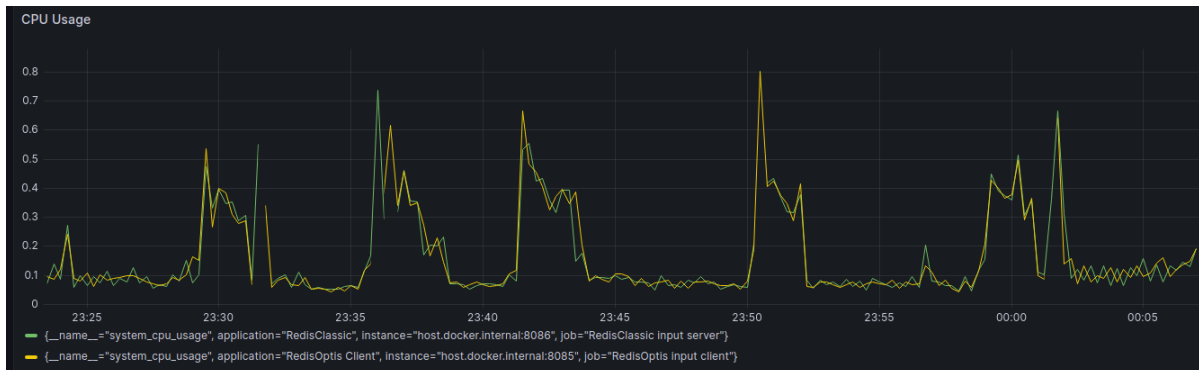
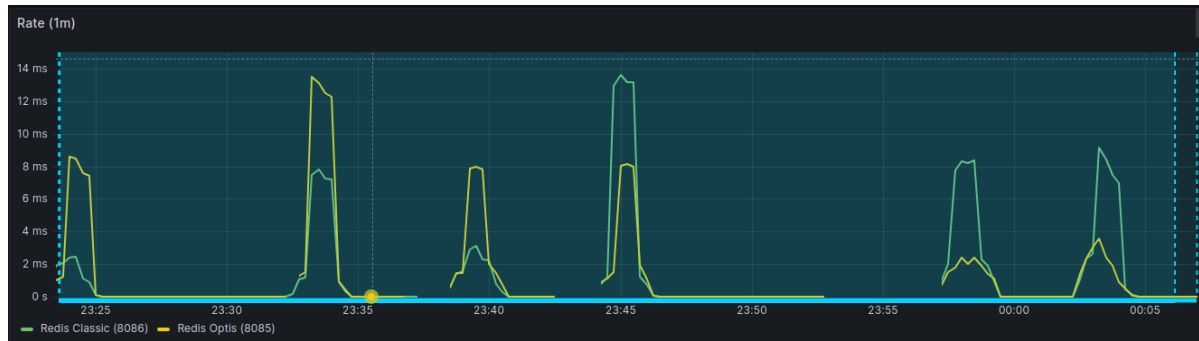


Results & Metrics

- 📊 Cache Miss Count (critical performance metric)
- 📊 Rate (Total Retrieval Time / No. of retrievals)
- 📊 Used Memory vs Max Memory
- 📊 CPU Usage

Experimental Evaluation

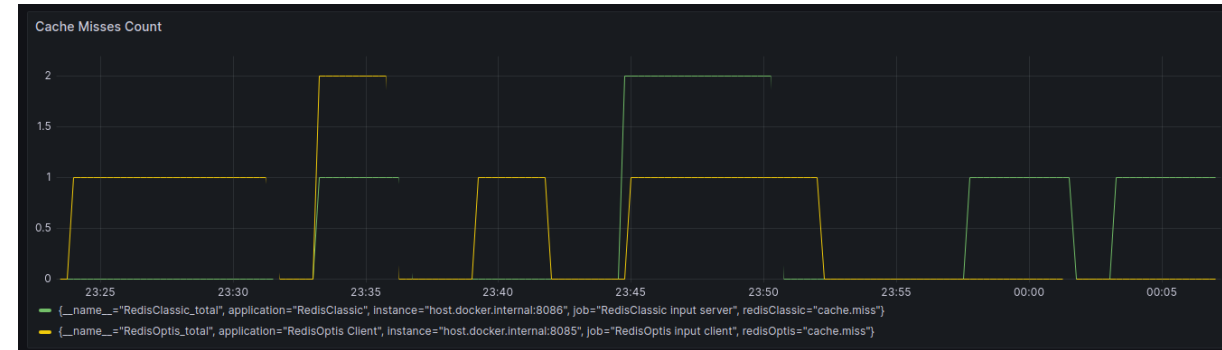
Random Scenario



HLRU with AT = 1

Data Access Pattern: No

Object Retrieval Pattern: Random



HRLU CM: {1, 2, 1, 1, 0, 0} = 5

LRU CM: {0, 1, 0, 2, 1, 1} = 5

Same Results



ΟΠΑ
ΑΥΕΒ

Experimental Evaluation

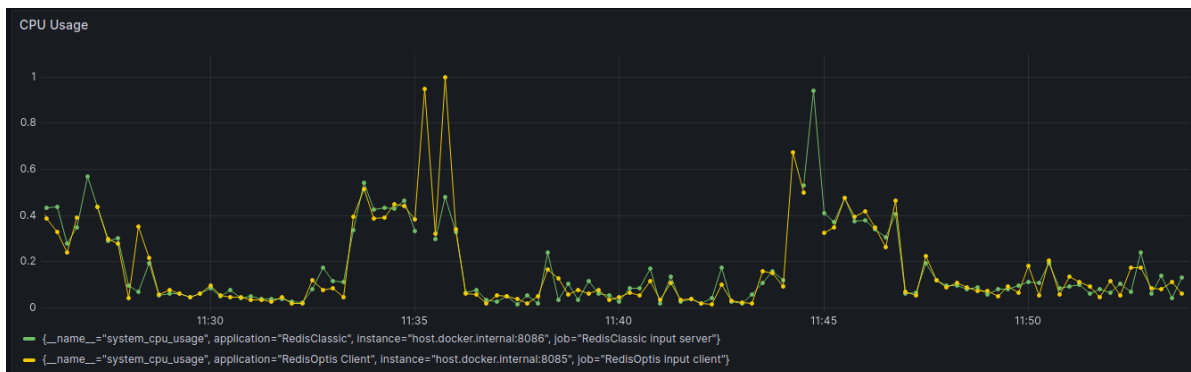
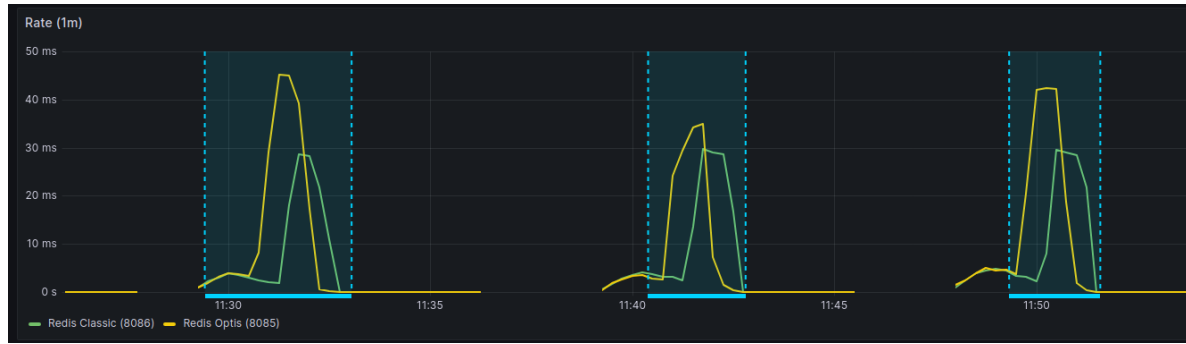
LRU-Favoured Scenario with $f > 1$



HLRU with $AT = 10$

Data Access Pattern: All data $f > 1$

Object Retrieval Pattern: Objects with increased frequency are not retrieved according to their popularity



HLRU Cache Misses: 21

LRU Cache Misses: 15

HLRU



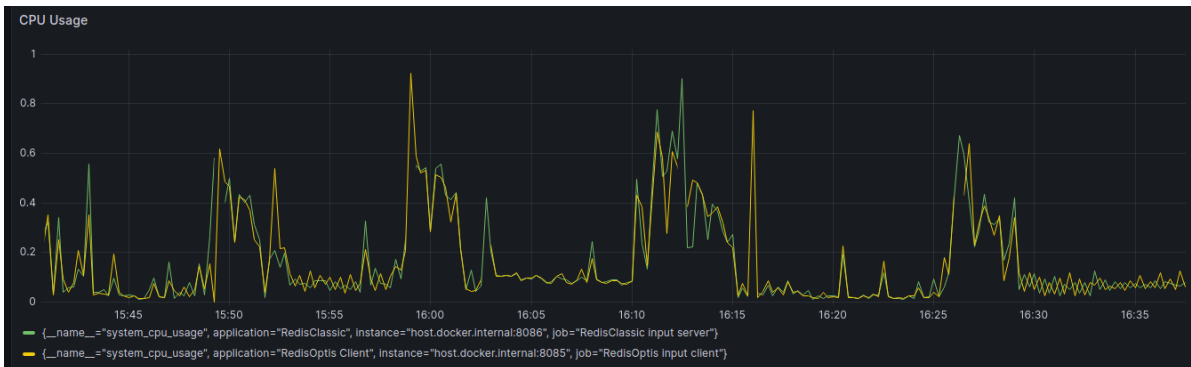
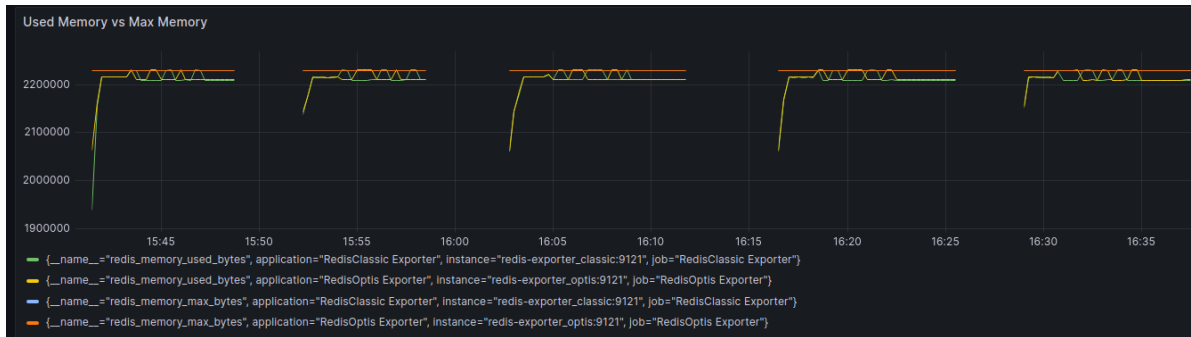
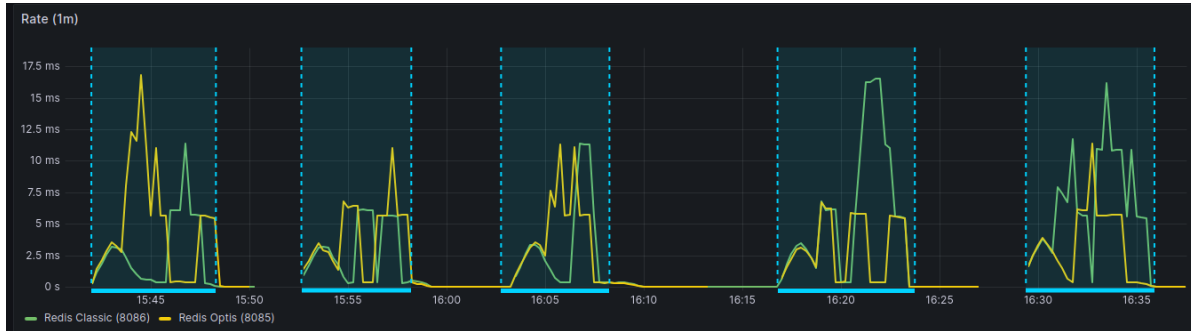
+40% CM



UAS
AUEB

Experimental Evaluation

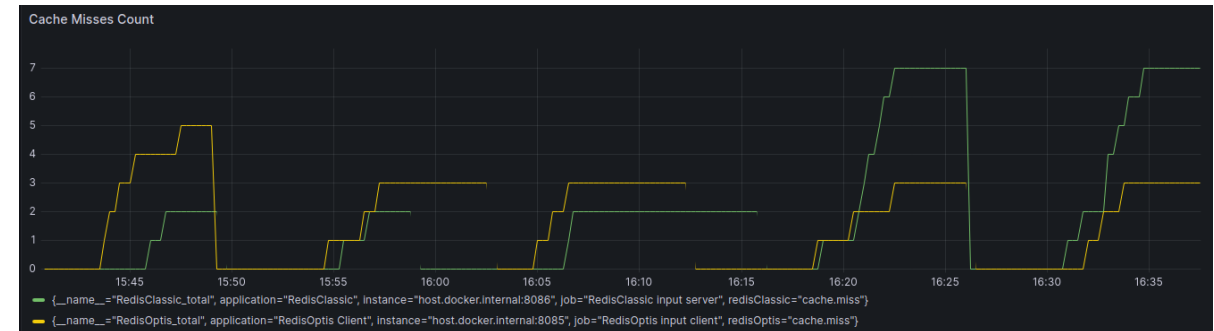
LRU-Favoured Scenario with $f = 1$



HLRU with AT = 10

Data Access Pattern: All data $f = 1$

Object Retrieval Pattern: LRU-favoured retrieval



Frequency does not constitute a criterion determining importance

HRLU Cache Misses: 17
LRU Cache Misses: 19

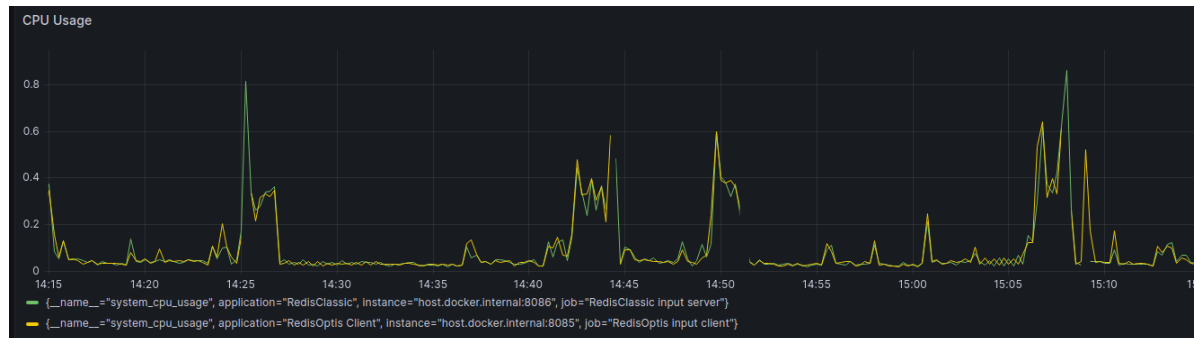
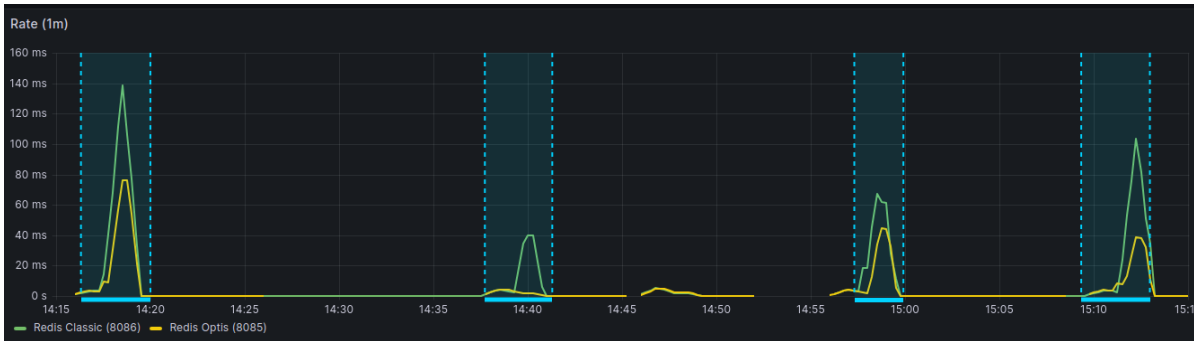
Same results



ΟΠΑ
AUEB

Experimental Evaluation

Hot & Cold



HLRU with AT = 5

Data Access Pattern: Hot and Cold Subset
Object Retrieval Pattern: According to the popularity of the item



HRLU Cache Misses: {15, 0, 8, 8} = 31

LRU Cache Misses: {27, 7, 14, 17} = 65

LRU → +110% CM

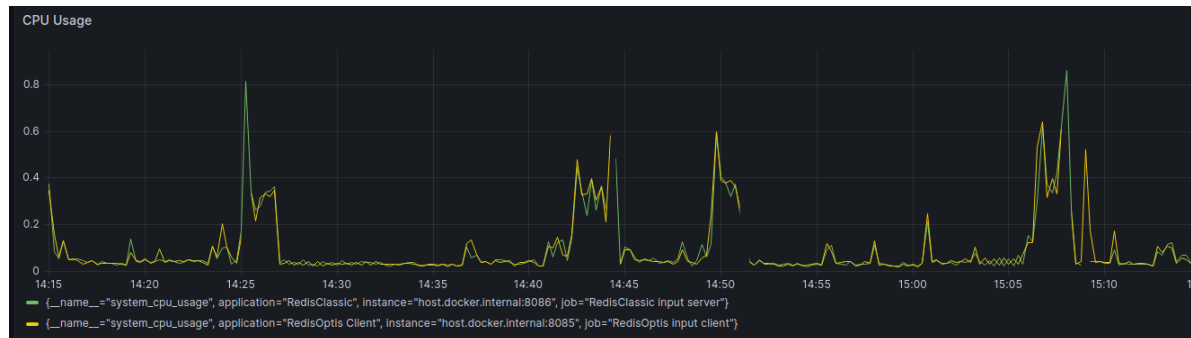
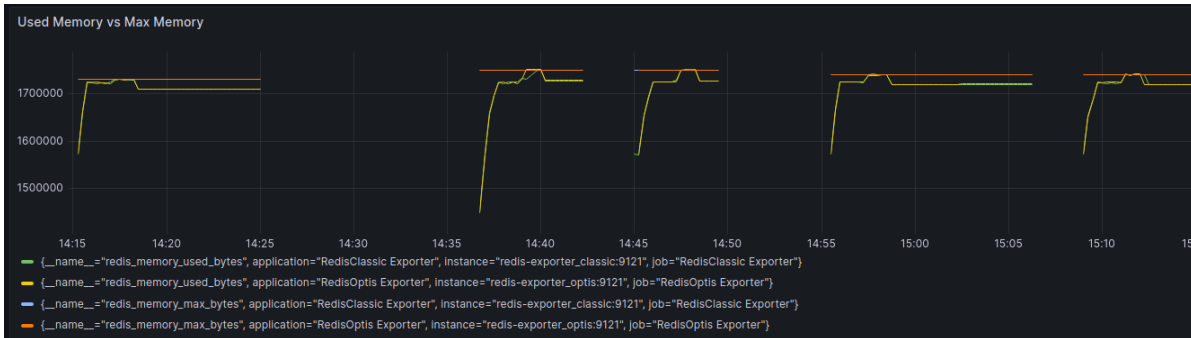
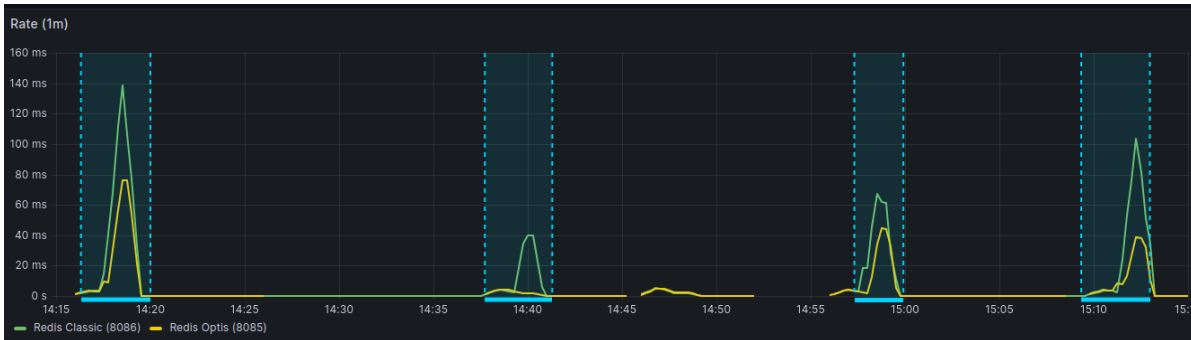
HRLU → -52.3% CM



ΟΠΑ
AUEB

Experimental Evaluation

Scan



HLRU with AT = 5

Data Access Pattern: Items have different attributes and popularity

Object Retrieval Pattern: Scanning of items



HLRU Cache Misses: {13, 11, 2, 2} = 28

LRU Cache Misses: {22, 20, 6, 3} = 51

LRU → **+82.1% CM**

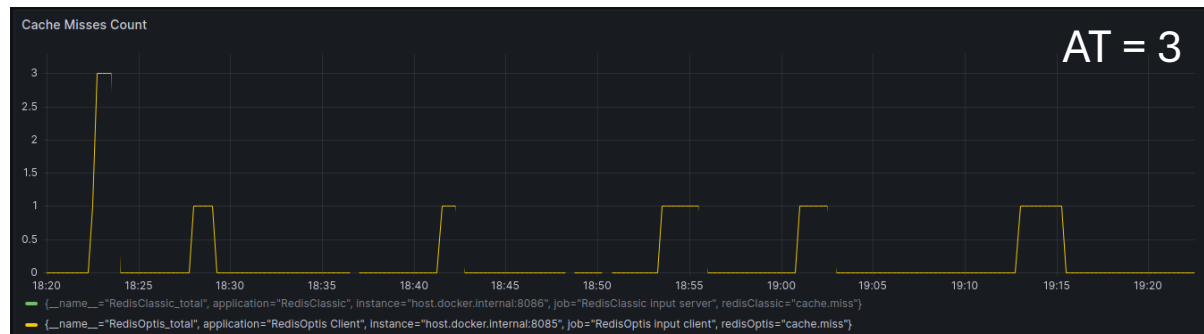
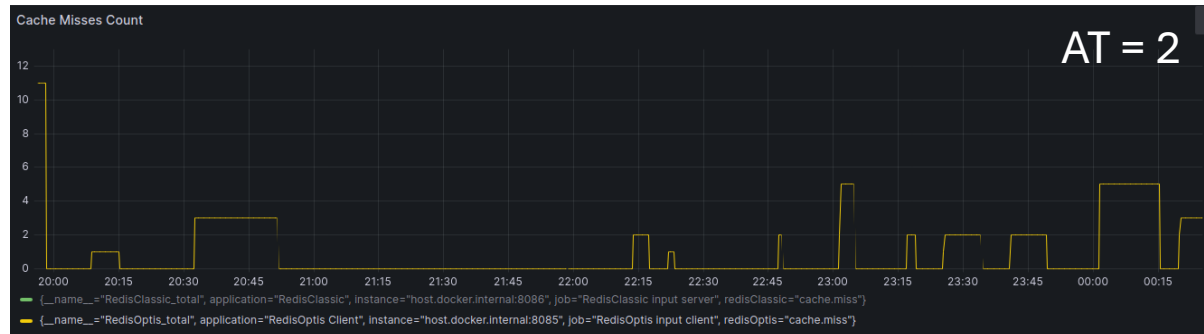
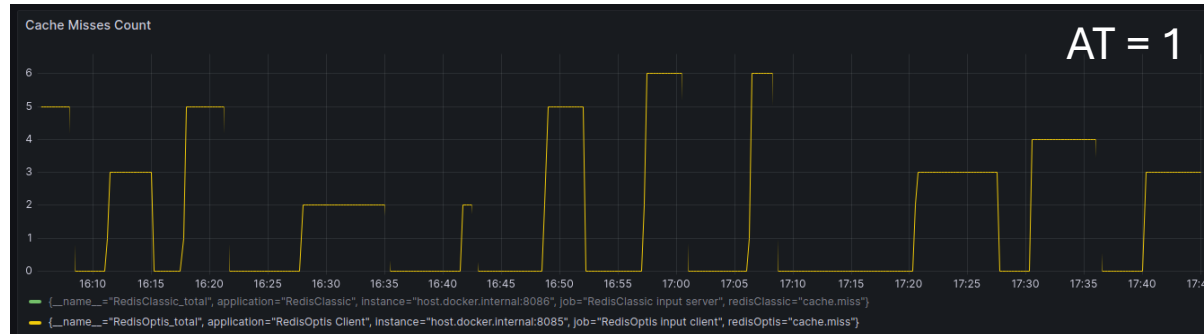
HLRU → **-45.1% CM**



ΟΡΠΑ
ΑΥΕΒ

Experimental Evaluation

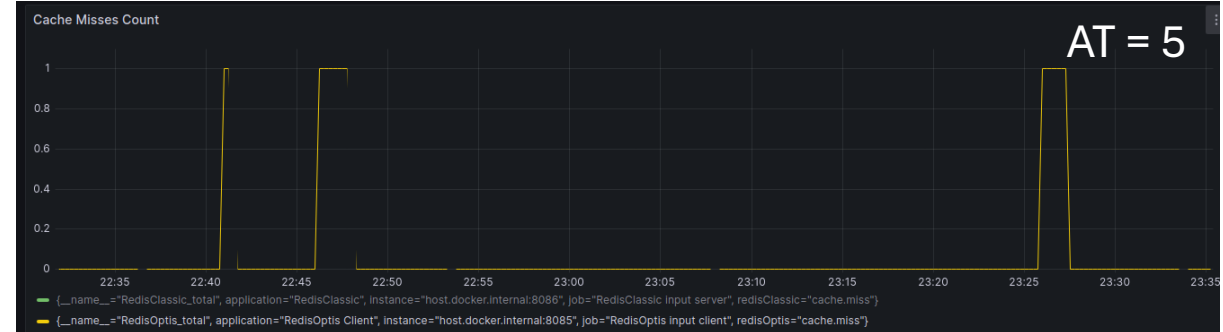
HLRU (AT = {1, 2, 3, 4, 5})



Frequency: $F = 5$

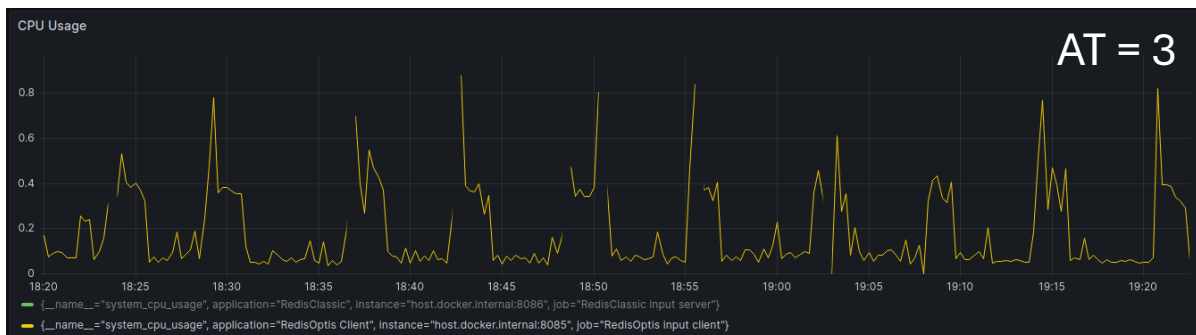
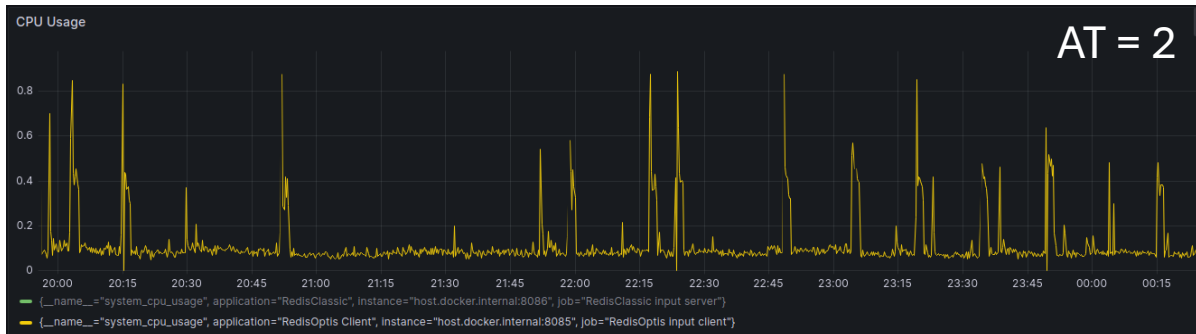
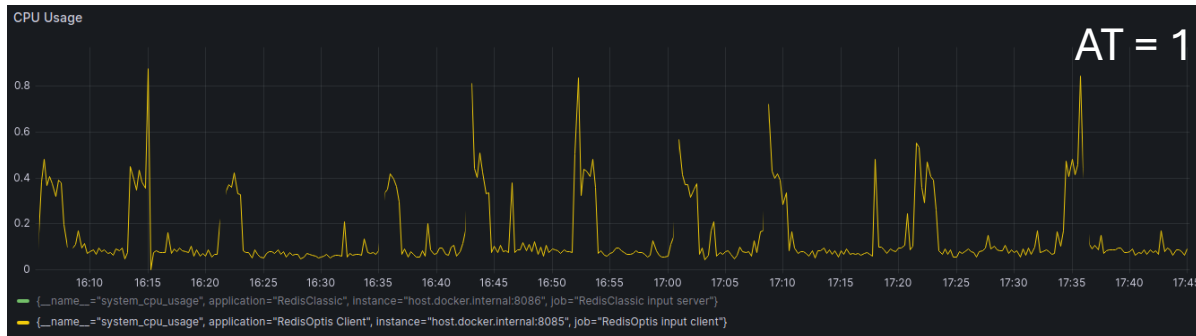
Data Access Pattern: Items called 5 times more and less recently than others.

Object Retrieval Pattern: According to access pattern



Experimental Evaluation

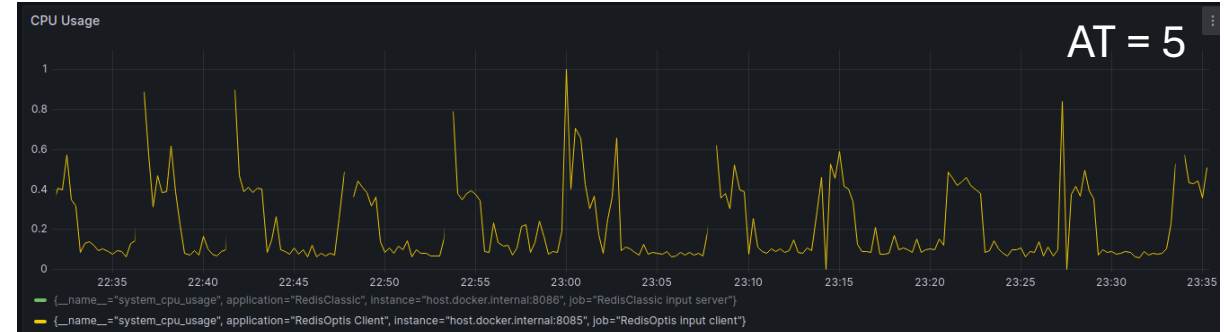
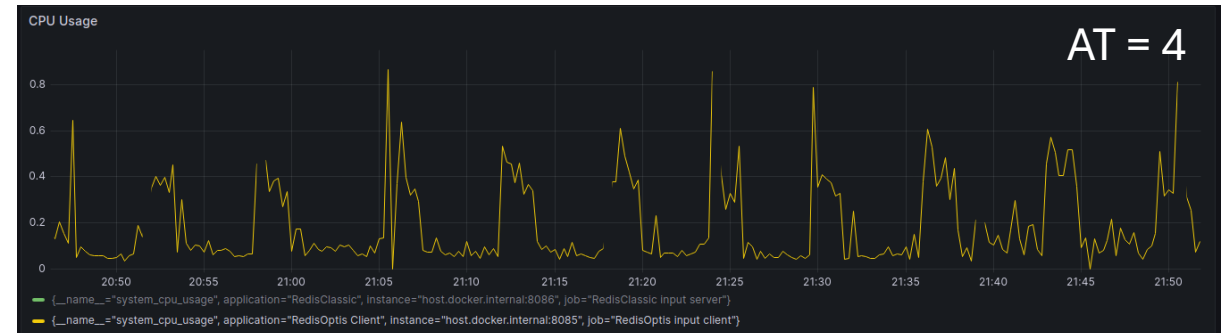
HLRU ($AT = \{1, 2, 3, 4, 5\}$)



Frequency: $F = 5$

Data Access Pattern: Items called 5 times more and less recently than others.

Object Retrieval Pattern: According to access pattern



Experimental Evaluation

HLRU (AT = {1, 2, 3, 4, 5}) - Memory Overhead

HLRU

- 1 byte for frequency
- 3 bytes for each entry in dynamic AT table

LRU

LRU algorithm includes *lru* field occupying 3 bytes, which is not used in our implementation, it is excluded from the calculations.

Μνήμη Ανά Αντικείμενο			
Τιμές AT	Μικρά Αντικείμενα	Μεσαία Αντικείμενα	Μεγάλα Αντικείμενα
AT = 1	$512 + 1 =$ 513 bytes	$5,120 + 1 =$ 5,121 bytes	$5,000,000 + 1 =$ 5,000,001 bytes
AT = 2	$512 + 1 + 3 =$ 516 bytes	$5,120 + 1 + 3 =$ 5,124 bytes	$5,000,000 + 1 + 3 =$ 5,000,004 bytes
AT = 3	$512 + 1 + 6 =$ 519 bytes	$5,120 + 1 + 6 =$ 5,127 bytes	$5,000,000 + 1 + 6 =$ 5,000,007 bytes
AT = 4	$512 + 1 + 9 =$ 522 bytes	$5,120 + 1 + 9 =$ 5,130 bytes	$5,000,000 + 1 + 9 =$ 5,000,010 bytes
AT = 5	$512 + 1 + 12 =$ 525 bytes	$5,120 + 1 + 12 =$ 5,133 bytes	$5,000,000 + 1 + 12 =$ 5,000,013 bytes

Experimental Evaluation

HLRU (AT = {1, 2, 3, 4, 5}) – Trade-off

Cache Misses / Εκτέλεση	AT = 1	AT = 2	AT = 3	AT = 4	AT = 5
1η	3	0	3	0	0
2η	5	1	1	0	1
3η	2	4	0	0	1
4η	2	0	1	0	0
5η	5	2	0	1	0
6η	6	1	1	2	0
7η	6	1	1	0	0
8η	3	5	0	1	0
9η	4	2	0	2	1
10η	3	2	1	0	0
Σύνολο	39	18	8	6	3
Μέσος Όρος Cache Miss	$\left(\frac{39}{10}\right) = 3.9$	$\left(\frac{18}{10}\right) = 1.8$	$\left(\frac{8}{10}\right) = 0.7$	$\left(\frac{6}{10}\right) = 0.6$	$\left(\frac{3}{10}\right) = 0.3$
Ποσοστό Αύξησης Μνήμης (Worst Case)	0.195%	0.781%	1.367%	1.953%	2.539%
Ποσοστό Αύξησης Μνήμης (Best Case)	0.00002%	0.00008%	0.00014%	0.0002%	0.00026%

Experimental Evaluation

LRU vs HLRU

Cache Misses

Random Scenario & Simple access patterns

Same performance

Specific scenarios – only recency matters

HLRU → +40% CM

Locality- & Frequency-oriented Scenarios

HLRU(Hot & Cold) → -52.3% CM

HLRU (Scan) → -45.1% CM

CPU Usage

Same

HLRU (AT = {1, 2, 3, 4, 5})

CM/Memory Trade-off

Increasing Memory (AT) - Decreases CM



Minimum AT = {1, 2}

- Critical Memory
- Not high chance to access same data
- Cache should always have more recent data

Maximum AT = {5}

- Memory not so restricting factor
- Workload with high locality
- Intolerant to CM & Restricting Latency

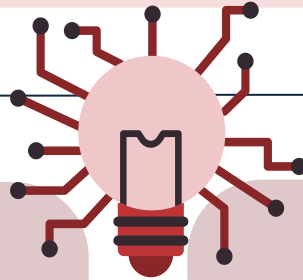
What about AT > 5?



ΟΠΑ
ΑΥΕΒ

Limitations

Limitations = room for improvement!



Limited Deductions in VM

Different Architectural Setups

- Multi-node architectures
- Remote Cache

Different Workloads



No universally best value for AT

Implementation of Redis client

- Track access pattern
- Correct choice and Adjustment of AT
- Adjust Window Weights



Questions?

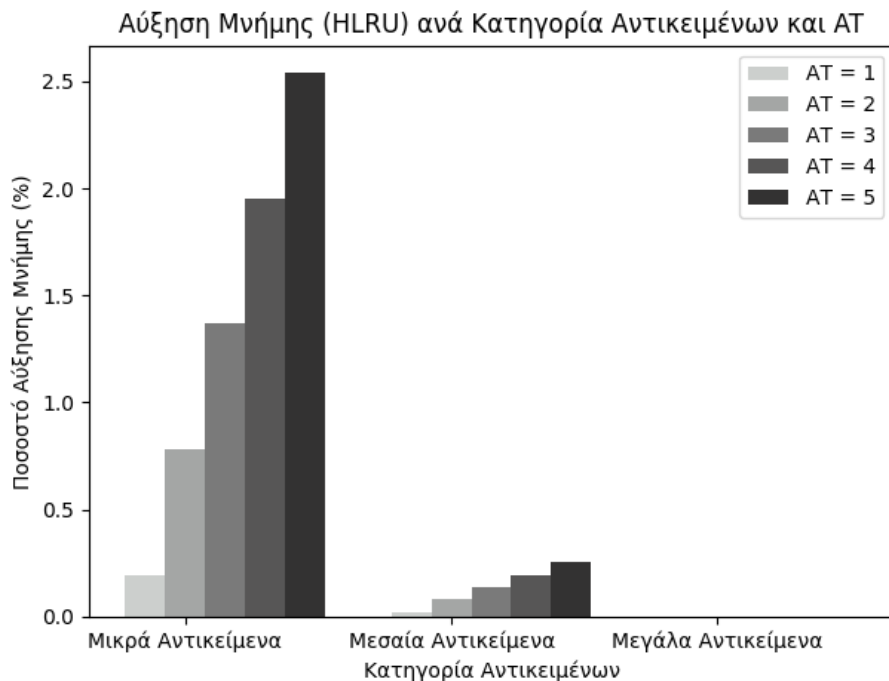
Thank you...!

Appendix

Μνήμη για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος			
Αντικείμενα που χωράνε σε 2 GB	Μικρά 4.000.000 αντικείμενα	Μεσαία 400.000 αντικείμενα	Μεγάλα 409 αντικείμενα
AT = 1	4.000.000 × 513 bytes = 2.052.000.000 bytes	400.000 × 5.121 bytes = 2.048.400.000 bytes	409 × 5.000.001 bytes = 2.045.000.409 bytes
AT = 2	4.000.000 × 516 bytes = 2.064.000.000 bytes	400.000 × 5.124 bytes = 2.049.600.000 bytes	409 × 5.000.004 bytes = 2.045.001.636 bytes
AT = 3	4.000.000 × 519 bytes = 2.076.000.000 bytes	400.000 × 5.127 bytes = 2.050.800.000 bytes	409 × 5.000.007 bytes = 2.045.002.863 bytes
AT = 4	4.000.000 × 522 bytes = 2.088.000.000 bytes	400.000 × 5.130 bytes = 2.052.000.000 bytes	409 × 5.000.010 bytes = 2.045.004.090 bytes
AT = 5	4.000.000 × 525 bytes = 2.100.000.000 bytes	400.000 × 5.133 bytes = 2.053.200.000 bytes	409 × 5.000.013 bytes = 2.045.005.317 bytes

Ποσοστό Αύξησης Μνήμης για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος			
Αντικείμενα που χωράνε σε 2GB	Μικρά αντικείμενα	Μεσαία αντικείμενα	Μεγάλα αντικείμενα
AT = 1	$\frac{(2,052,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.195\%$	$\frac{(2,048,400,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.0195\%$	$\frac{(2,045,000,409 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00002\%$
AT = 2	$\frac{(2,064,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.781\%$	$\frac{(2,049,600,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.0781\%$	$\frac{(2,045,001,636 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00008\%$
AT = 3	$\frac{(2,076,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 1.367\%$	$\frac{(2,050,800,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.1367\%$	$\frac{(2,045,002,863 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00014\%$
AT = 4	$\frac{(2,088,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 1.953\%$	$\frac{(2,052,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.1953\%$	$\frac{(2,045,004,090 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.0002\%$
AT = 5	$\frac{(2,100,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 2.539\%$	$\frac{(2,053,200,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.2539\%$	$\frac{(2,045,005,317 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00026\%$

Appendix



Αξιολόγηση HLRU βάσει AT			
Τιμή AT	Πότε είναι χρήσιμη;	Πλεονεκτήματα	Μειονεκτήματα
Minimum AT = {1, 2}	■ Περιορισμένη μνήμη ■ Όταν τα δεδομένα δεν έχουν υψηλή πιθανότητα να ξαναζητηθούν (low locality) ■ Όταν η cache πρέπει να έχει πάντα τα πιο πρόσφατα δεδομένα (real-time updates)	■ Χαμηλότερη χρήση μνήμης (συγκριτικά)	■ Υψηλότερο ποσοστό cache misses ■ Μικρότερο caching όφελος σε workloads με υψηλό locality
Medium	■ Γενικά ισορροπημένη επιλογή ■ Όταν θέλουμε σημαντική μείωση cache misses χωρίς υπερβολική χρήση μνήμης	■ Μειώνει δραστικά τα cache misses	■ Ελαφρώς μεγαλύτερη κατανάλωση μνήμης σε σχέση με AT=1
Maximum AT = 5	■ Η μνήμη δεν είναι περιοριστικός παράγοντας ■ Όταν το workload έχει υψηλό locality (τα ίδια δεδομένα ζητούνται ξανά & ξανά) ■ Όταν ανεχόμαστε ελάχιστα cache misses και χρειαζόμαστε χαμηλό latency	■ Λιγότερα cache misses ■ Ιδανικό για high-performance workloads	■ Υψηλότερη χρήση μνήμης ■ Περιττή χρήση μνήμης σε μη ανταγωνιστικά περιβάλλοντα ή workloads με χαμηλό locality

Appendix

Σparse Παράδειγμα Υπολογισμού Παραθύρων Ηλικίας

Δεδομένα

- Αντικείμενο A_1
- Age of access times: [1, 20, 30, 100]
- Αντικείμενο A_2
- Age of access times: [1, 20, 30, 80]

Υπολογισμοί

- $global_{min} = \min(\min_{A_1}(1, 20, 30, 100), \min_{A_2}(1, 20, 30, 80)) = 1$
- $global_{max} = \min(\min_{A_1}(1, 20, 30, 100), \min_{A_2}(1, 20, 30, 80)) = 100$
- $mean = \frac{(1+20+30+100+1+20+30+80)}{8} = 35.25$
- $sttdev = \sqrt{\frac{(1-35.25)^2 + (20-35.25)^2 + \dots + (30-35.25)^2 + (80-35.25)^2}{8}} = \sqrt{\frac{9061.5}{8}} \approx 33.65$
- $mean - sttdev = 35.25 - 33.65 = 1.6$
- $mean + sttdev = 35.25 + 33.65 = 68.9$

Τελικά Παράθυρα

- 1° παράθυρο: [1, 1.6)
- 2° παράθυρο: [1.6, 35.25)
- 3° παράθυρο: [35.25, 68.9)
- 4° παράθυρο: [68.9, 100]

Appendix

Dense Παράδειγμα Υπολογισμού Παραθύρων Ηλικίας

Δεδομένα

- Αντικείμενο A_1
- Age of access times: [1, 2, 3, 7]
- Αντικείμενο A_2
- Age of access times: [1, 2, 3, 5]

Υπολογισμοί

- $global_{min} = \min(\min_{A_1}(1, 2, 3, 7), \min_{A_2}(1, 2, 3, 5)) = 1$
- $global_{max} = \min(\min_{A_1}(1, 2, 3, 7), \min_{A_2}(1, 2, 3, 5)) = 7$
- $mean = \frac{(1+2+3+7+1+2+3+5)}{8} = 3$
- $sum_{ages_{squared}} = 1^2 + 2^2 + 3^2 + 7^2 + 1^2 + 2^2 + 3^2 + 5^2 = 102$
- $sttdev = \sqrt{\frac{102}{8} - 3^2} = \sqrt{12.75 - 9} \approx 1.93$
- $mean - sttdev = 3 - 1.93 = 1.07$
- $mean + sttdev = 3 + 1.93 = 4.93$

Τελικά Παράθυρα

- 1^ο παράθυρο: [1, 1.07)
- 2^ο παράθυρο: [1.07, 3)
- 3^ο παράθυρο: [3, 4.93)
- 4^ο παράθυρο: [4.93, 7]

Appendix

Παράδειγμα Υπολογισμού Τελικού Score

Δεδομένα

- $WINDOW_WEIGHTS = \{0.4, 0.3, 0.2, 0.15\}$
- $NOW = 10000$
- Αντικείμενο A_1
 - Access Times: 9000, 3000, 2000
 - Frequency: 3
- Αντικείμενο A_2
 - Access Times: 1500, 2000, 3000
 - Frequency: 3

Υπολογισμοί

- $ages_{A1} = [1000, 7000, 9000]$
- $ages_{A2} = [8500, 8000, 7000]$
- $global_{min} = \min(\min_{A1}(1000, 7000, 9000), \min_{A2}(8500, 8000, 7000)) = \mathbf{1000}$
- $global_{max} = \min(\min_{A1}(1000, 7000, 9000), \min_{A2}(8500, 8000, 7000)) = \mathbf{9000}$
- $sum_{ages} = 1000 + 7000 + 8000 + 8500 + 8000 + 7000 = \mathbf{39500}$
- $mean = \frac{39500}{6} \cong \mathbf{6583}$
- $sum_{ages_{squared}} = 1000^2 + 7000^2 + 8000^2 + 8500^2 + 8000^2 + 7000^2$
 $= 1000000 + 49000000 + 64000000 + 72250000 + 64000000$
 $+ 49000000 = 316250000$
- $sttdev = \sqrt{\frac{316250000}{6} - \left(\frac{39500}{6}\right)^2} = \sqrt{52708333.33 - 43383333.33} \approx \mathbf{3054}$
- $windows = [1000, 3529, 6583, 9637]$
- $score_{A1} = (1 \times 0.4) + (0 \times 0.3) + (2 \times 0.2) + (0 \times 0.15) + 0.01 \times 3 =$
 $0.4 + 0 + 0.4 + 0 + 0.03 = \mathbf{0.83}$
- $score_{A2} = (0 \times 0.4) + (0 \times 0.3) + (2 \times 0.2) + (1 \times 0.15) + 0.01 \times 3 = 0 +$
 $0 + 0.4 + 0.15 + 0.03 = \mathbf{0.58}$

Συμπέρασμα

Το αντικείμενο A_1 έχει το υψηλότερο score, και θεωρείται πιο σημαντικό σε σχέση με το A_2 , Αυτό οφείλεται στο γεγονός ότι η πρόσφατη πρόσβαση στο A_1 , επηρέασε θετικά το score του.

Appendix

```
typedef struct {
    unsigned access_time: LRU_BITS; // Use exactly LRU_BITS for each entry
} AccessTime;

struct redisObject {
    unsigned type:4;
    unsigned encoding:4;
    // unsigned lru:LRU_BITS; /* LRU time (relative to global lru_clock) */
    int refcount;
    void *ptr;
    AccessTime access_times[HISTORY_SIZE]; /* Dynamic Array */
    unsigned num_accesses:ACCESS_BITS; /* Number of accesses */
};
```

*Δημιουργία HLRU variables στο redisObject
(access_times and num_accesses)*

```
# Eviction policy: allkeys-lru-histogram
maxmemory-policy allkeys-lru-histogram
# Define how many access times should be kept for each item.
## This should be a value between 1 and 5.
history-count 5
# Defines the weights of the windows created during the eviction phase for building the histogram.
## A descending array of numbers; the higher values indicate that the first windows
## (and hence the most recently used items) are more important.
histogram-window-weights 0.4 0.3 0.2 0.15
```

Δυναμικό Configuration
(history-count, histogram-window-weights)

Appendix

```
void calculateStatistics(dictEntry **samples, int sample_count, unsigned long long *global_min,
                      unsigned long long *global_max, double *mean, double *stddev) {
    unsigned long long current_time = LRU_CLOCK(), sum = 0, sum_of_squares = 0, sum_of_accesses = 0, count = 0;
    *global_min = ULLONG_MAX;
    *global_max = 0;

    for (int j = 0; j < sample_count; j++) {
        dictEntry *de = samples[j];
        sds key = dictGetKey(de);
        robj *o = dictGetVal(de);
        unsigned long long obj_min = ULLONG_MAX;
        unsigned long long obj_max = 0;

        for (unsigned int i = 0; i < o->num_accesses; i++) {
            unsigned long long age = current_time - o->access_times[i];
            // Update object-level min and max & Update global min and max
            if (age < obj_min) obj_min = age;
            if (age > obj_max) obj_max = age;
            if (age < *global_min) *global_min = age;
            if (age > *global_max) *global_max = age;

            sum += age;
            sum_of_squares += age * age;
            count++;
        }
    }

    // Compute mean and standard deviation
    if (count > 0) {
        *mean = (double)sum / count;
        *stddev = sqrt((double)sum_of_squares / count - (*mean) * (*mean));
    } else {
        *mean = 0.0;
        *stddev = 0.0;
    }
}
```

```
void defineWindowBoundaries(double *window_boundaries, unsigned long long global_min,
                          unsigned long long global_max, double mean, double stddev) {

    window_boundaries[0] = global_min;

    window_boundaries[1] = fmax(global_min, mean - stddev); // No less than global_min
    window_boundaries[2] = fmax(global_min, fmin(global_max, mean)); // Clamped between min and max
    window_boundaries[3] = fmin(global_max, mean + stddev); // No more than global_max

    window_boundaries[4] = global_max;

}
```

Ορισμός Δυναμικών Παραθύρων Ιστογράμματος.

Υπολογισμός Στατιστικών Δεδομένων
(Mean, Standard Deviation)

Appendix

Υπολογισμός τελικού score.

```
double estimateObjectEvictionScore(robj *o, double *window_boundaries, sds key) {
    unsigned long long current_time = LRU_CLOCK();
    const double WINDOW_WEIGHTS[NUM_WINDOWS] = {
        WINDOW_WEIGHT_1,
        WINDOW_WEIGHT_2,
        WINDOW_WEIGHT_3,
        WINDOW_WEIGHT_4,
        WINDOW_WEIGHT_5
    };

    int window_counts[NUM_WINDOWS+1] = {0};
    double score = 0.0;

    // Loop through each access time stored for the object
    for (unsigned int i = 0; i < o->num_accesses; i++) {
        unsigned long long age = current_time - o->access_times[i];
        // Determine the window each access time belongs to based on dynamic boundaries
        for (int j = 1; j < NUM_WINDOWS+1; j++) {
            if (age <= window_boundaries[j]) {
                window_counts[j]++;
                break; // Exit the loop after determining the appropriate window
            }
        }
    }

    // Calculate weighted score
    for (int i = 1; i < NUM_WINDOWS; i++) {
        score += (double)window_counts[i] * WINDOW_WEIGHTS[i];
    }

    double tie_breaker = 0.01 * o->num_accesses; // Small tie-breaking factor based on num_accesses
    score += tie_breaker;
    return -score;
}
```